

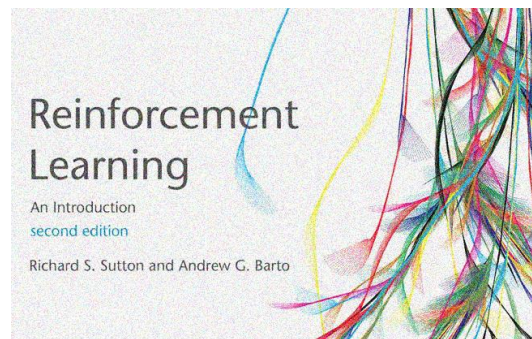


دانشگاه اصفهان

دانشکده مهندسی کامپیوتر - گروه هوش مصنوعی و رباتیک

گزارش تمرین دوم - محیط شبکه‌ای تصادفی

Stochastic Cliff Walking Env



پدیدآورنده:

محمد امین کیانی

۴۰۴۳۶۴۴۰۰۸

دانشجوی کارشناسی ارشد، دانشکده‌ی کامپیوتر، دانشگاه اصفهان، اصفهان،
Aminkianiworkeng@gmail.com

استاد درس: دکتر کارشناس

نیمسال دوم تحصیلی ۱۴۰۴-۰۵

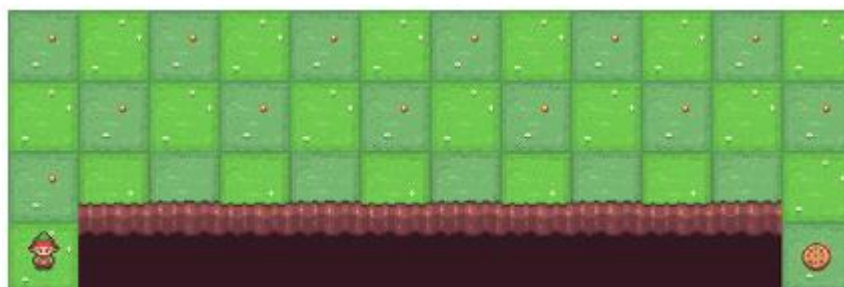
فهرست مطالب

۳	مستندات.....
۳	۰- مسئله و تحلیل کلی آن:.....
۷	۱- بخش اول: Monte Carlo.....
۷	سوال (۱): تولید اپیزود (Generate Episode).....
۸	سوال (۲): الگوریتم First Visit Monte Carlo برای سیاست soft-epsilon.....
۱۲	سوال (۳): رسم نمودار پایداری یادگیری (۱۰ اجرای مستقل).....
۱۳	سوال (۴): پیش‌بینی ارزش با نمونه‌برداری اهمیت (OIS vs WIS).....
۲۰	۲- بخش دوم: تفاوت TD و MC.....
۲۰	گام صفر: محاسبه ارزش واقعی $V_{\pi}(s)$ با Iterative Policy Evaluation.....
۲۱	سوال (۱): مقایسه Every-visit MC و $TD(0)$
۲۴	سوال (۲): پیش‌بینی n-step TD و موازنه بایاس-واریانس:.....
۲۵	سوال (۳) - امتیازی: هم‌ارزی n-step TD و مونت کارلو:.....
۳۲	۳- بخش سوم: کنترل TD.....
۳۲	سوال (۱ و ۲): SARSA ، Q-Learning و Expected SARSA.....
۳۵	سوال (۳): Double Q-Learning و Maximization Bias.....
۳۷	سوال (۴): مقایسه جامع کنترل TD و MC:.....
۴۴	۴- مراجع.....

مستندات

۰- مسئله و تحلیل کلی آن:

محیط این تمرین نسخه‌ای تصادفی از مسئله کلاسیک Cliff Walking است. این محیط شامل یک شبکه مستطیلی ۴ در ۱۲ است. عامل از خانه ۳۶ (گوشه پایین چپ) شروع کرده و باید به خانه ۴۷ (گوشه پایین راست) برسد. خانه‌های ردیف پایین بین شروع و هدف (۳۷ تا ۴۶)، صخره (Cliff) هستند که ورود به آن‌ها پاداش ۱۰۰- داشته و عامل را مجدداً به نقطه شروع برمی‌گرداند بدون اینکه اپیزود تمام شود. هر گام معمولی پاداش ۱- دارد و اگر زمان از ۲۰۰ گام بگذرد، پاداش ۵۰- اعمال شده و اپیزود به اتمام می‌رسد. به دلیل نرخ لغزش ($Slip Rate = 0.1$)، عامل با احتمال ۱۰ درصد به جای جهت اصلی، به یکی از جهت‌های عمود بر آن پرتاب می‌شود.



در نسخه‌ی stochastic، نتیجه‌ی action کاملاً قطعی نیست یعنی اگر مثلاً بگوییم «برو راست»، ممکن است محیط کمی سر بخورد و به جای راست، بالا یا پایین هم برود. پس عامل همیشه دقیقاً به همان خانه‌ای که می‌خواهد نمی‌رسد اما در محیط deterministic اگر خیلی دقیق راه برویم، می‌توانیم سریع‌تر از کنار پرتگاه رد شویم. پس اینجا یک مسیر امن‌تر ولی طولانی‌تر، ممکن است بهتر باشد.

چرا این محیط برای مقایسه‌ی الگوریتم‌ها خیلی خوب است؟

چون در این محیط، الگوریتم‌ها رفتارهای متفاوتی نشان می‌دهند:

- بعضی‌ها حریصانه و خطرپذیرند .

- بعضی‌ها محتاط‌ترند .
 - بعضی‌ها اثر **stochasticity** را خوب در نظر می‌گیرند .
 - بعضی‌ها در تخمین ارزش دچار **overestimation** می‌شوند .
- به همین دلیل این محیط برای فهم:

• MC

- تا پایان episode صبر کن،
- بعد بازده واقعی را حساب کن،
- بعد ارزش را به‌روزرسانی کن.
- مزیت: مدل محیط لازم ندارد.
- عیب: باید episode تمام شود و variance زیاد است.

• TD

- صبر نکن تا episode تمام شود،
- از همان قدم بعدی برای به‌روزرسانی استفاده کن.
- فرمول TD (۰) برای حالت: $V(s) \leftarrow V(s) + \alpha [r + \gamma V(s') - V(s)]$
- TD از bootstrap استفاده می‌کند؛ یعنی به تخمین خودش تکیه می‌کند.

• On-policy / Off-policy

- On-policy : سیاستی که داده تولید می‌کند، همان سیاستی است که می‌خواهیم یاد بگیریم. مثال: SARSA
- Off-policy : سیاستی که داده تولید می‌کند، با سیاستی که می‌خواهیم یاد بگیریم فرق دارد. مثال: Q-learning

• SARSA

- یک الگوریتم on-policy است.
- فرمول: $Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma Q(s',a') - Q(s,a)]$
- یعنی: در state فعلی action را انجام می‌دهیم، reward می‌گیریم، به state بعدی می‌رویم، action بعدی را هم با همان policy فعلی انتخاب می‌کنیم و با همان action بعدی update می‌کنیم.
- چرا در **Cliff Walking** مهم است؟ چون SARSA سیاستی را یاد می‌گیرد که واقعاً با آن رفتار می‌کند. اگر policy رفتاری greedy- ϵ باشد، SARSA می‌فهمد که همیشه امکان دارد در مسیر کمی اشتباه رخ دهد. پس در محیط stochastic معمولاً مسیر امن‌تر را ترجیح می‌دهد. پس ممکن است گاهی بلغزیم، بهتر است نزدیک cliff نرویم.

• Q-learning

- یک الگوریتم off-policy است.
- فرمول: $Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$
- یعنی: با policy رفتارگر هر چیزی می‌توانیم داده جمع کنیم اما target را با greedy action می‌سازیم و این الگوریتم به سمت optimal policy می‌رود.
- چرا در **Cliff Walking** مهم است؟ Q-learning معمولاً مسیر نزدیک به cliff را ترجیح می‌دهد، چون از دید optimal greedy، سریع‌ترین مسیر ارزش بیشتری دارد ولی چون محیط stochastic است، این مسیر در عمل خطرناک‌تر می‌شود و ممکن است عامل چندبار سقوط کند و رفتارش تهاجمی‌تر و پرریسک‌تر باشد. پس اینجا بهترین مسیر تئوری را می‌خواهیم؛ اگر لیز خورد، مشکل خودش است.

• Expected SARSA

- بین SARSA و Q-learning است.
- فرمول target: $r + \gamma \sum_{a'} \pi(a'|s') Q(s', a')$
- یعنی به جای استفاده از یک action نمونه برداری شده یا max، از امید ریاضی روی action ها استفاده می کند.
- چرا بهتر است؟ variance کمتر از SARSA + خوش بینی بیش از حد Q-learning را ندارد و در محیط stochastic معمولاً رفتار متعادل تری دارد و فقط یک action بعدی را نمی بیند بلکه کل توزیع احتمالاتی action ها را لحاظ می کند.

• Double Q-learning

- برای حل maximization bias است.
- ایده: دو تخمین جدا از Q داشته باشیم، یکی برای انتخاب action و دیگری برای ارزیابی action و چون max روی یک تخمین نویزی، overestimation می سازد و Double Q-learning این مشکل را کم می کند. این جداسازی باعث می شود خوش بینی کاذب کمتر شود.

• Maximization Bias

- وقتی در Q-learning از max استفاده می کنیم، اگر تخمین ها کمی نویزی باشند، max معمولاً مقدارها را بیش از حد خوش بینانه انتخاب می کند. یعنی: از بین چند تخمین تقریباً مشابه، نویز باعث می شود یکی از آنها خیلی بزرگ تر دیده شود و الگوریتم فکر کند آن action بهتر از واقعیت است. این را overestimation می گویند.

بسیار عالی است.

MC update

$$V(s) \leftarrow V(s) + \alpha(G - V(s))$$

TD(0)

$$V(s) \leftarrow V(s) + \alpha[r + \gamma V(s') - V(s)]$$

SARSA

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)]$$

Q-learning

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Expected SARSA

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \sum_{a'} \pi(a'|s') Q(s', a') - Q(s, a) \right]$$

ابتدا یک سلول کد در pycharm ایجاد کردیم و کدهای مربوط به کتابخانه‌ها و کلاسی که دستیاران آموزشی فرستادند (StochasticCliffWalkingEnv) را در آن قرار دادیم و اجرا کردیم. سپس به توسعه‌ی آن در ادامه می‌پردازیم

۱- بخش اول: Monte Carlo

سوال (۱): تولید اپیزود (Generate Episode)

در این بخش باید تابعی بنویسیم که یک محیط و یک سیاست را دریافت کند و تا پایان یک اپیزود، مسیر (وضعیت‌ها، اکشن‌ها و پاداش‌ها) را تولید کند.

```
import numpy as np
import matplotlib.pyplot as plt

def generate_episode(pi, env, render=False):
    """
    states, actions, rewards
    """
```

```

states = []
actions = []
rewards = []

state, _ = env.reset()
done = False
truncated = False

while not (done or truncated):
    if render:
        env.render()

    # سیاست از فعلی وضعیت برای اکشن احتمالات دریافت
    action_probs = pi(state)
    # احتمالاتی توزیع اساس بر اکشن انتخاب
    action = np.random.choice(len(action_probs), p=action_probs)

    # محیط در اکشن انجام
    next_state, reward, done, truncated, _ = env.step(action)

    states.append(state)
    actions.append(action)
    rewards.append(reward)

    state = next_state

return states, actions, rewards

```

سوال (۲): الگوریتم First Visit Monte Carlo برای سیاست soft-epsilon

در اینجا باید الگوریتم کنترلی MC را برای یافتن سیاست بهینه پیاده‌سازی کنیم. در این روش همواره احتمال کوچکی (ϵ) برای انتخاب اکشن‌های تصادفی وجود دارد تا اکتشاف (Exploration) تضمین شود.

```

from collections import defaultdict

def make_epsilon_soft_policy(Q, epsilon, nA):
    """
    Q مقادیر اساس بر e-soft سیاست تابع یک ایجاد
    """
    def policy_fn(observation):
        A = np.ones(nA, dtype=float) * epsilon / nA
        best_action = np.argmax(Q[observation])
        A[best_action] += (1.0 - epsilon)
        return A
    return policy_fn

def mc_control_epsilon_soft(env, num_episodes, epsilon, gamma=0.99):
    """
    First-visit Monte Carlo Control با epsilon-soft
    """

```



```

nA = env.action_space.n
# (تصادفی یا) صفر صورت به Q اولیه مقداردهی
Q = defaultdict(lambda: np.zeros(nA))
# (وضعیت،) جفت برای شده دیده (Return) بازده‌های تمام نگهداری برای دیکشنری
(اکشن)
returns_sum = defaultdict(float)
returns_count = defaultdict(float)

# باید می بهبود یادگیری طول در که سیاستی
policy = make_epsilon_soft_policy(Q, epsilon, nA)

for ep in range(num_episodes):
    if (ep+1) % 1000 == 0:
        print(f"\rEpisode {ep+1}/{num_episodes}", end="")

        # اپیزود یک تولید
        states, actions, rewards = generate_episode(policy, env,
render=False)

        # ابتدا به اپیزود انتهای از (G) ها بازده محاسبه
        G = 0
        # در قبلاً (اکشن وضعیت،) جفت آیا که کنیم بررسی باید First-Visit
        خیر یا شده دیده اپیزود این
        visited_state_actions = set()

        for t in range(len(states)-1, -1, -1):
            state = states[t]
            action = actions[t]
            reward = rewards[t]

            G = gamma * G + reward
            state_action = (state, action)

            # (s, a) به اپیزود در که باری اولین فقط، First-Visit الگوریتم در
            دهیم می انجام را آپدیت رسم می
            if state_action not in zip(states[:t], actions[:t]):
                visited_state_actions.add(state_action)
                returns_sum[state_action] += G
                returns_count[state_action] += 1

            # میانگین رسانی روز به
            Q[state][action] = returns_sum[state_action] /
returns_count[state_action]
            # دارد ارجاع Q به چون شود می روز به خودکار طور به سیاست

        return Q, policy

# مسیر نمایش برای کمکی توابع
def get_path(env, policy)::
    """(تصادفی اکتشاف بدون) اکشن بهترین انتخاب با مسیر استخراج"""
    # می‌کنیم تولید Q به نسبت حریصانه کاملاً را مسیر تست، مرحله در
    def greedy_policy(state)::
        probs = np.zeros(env.action_space.n)
        best_a = np.argmax(policy(state)) # policy(state) gives e-soft probs,
max prob is greedy action
        probs[best_a] = 1.0
        return probs

```

```

states, _, _ = generate_episode(greedy_policy, env, render=False)
return states

def print_path_on_grid(env, path):
    nrow, ncol = env.nrow, env.ncol
    grid = np.array([[ '.' for _ in range(ncol)] for _ in range(nrow)])

    # صخره و هدف شروع، کردن مشخص
    for r in range(nrow):
        for c in range(ncol):
            if env.desc[r, c] == b'S': grid[r, c] = 'S'
            elif env.desc[r, c] == b'G': grid[r, c] = 'G'
            elif env.desc[r, c] == b'C': grid[r, c] = 'C'

    # مسیر رسم
    for s in path[:-1]:
        r, c = s // ncol, s % ncol
        if grid[r, c] not in ['S', 'G']:
            grid[r, c] = '*'

    print("\n" + "\n".join([''.join(row) for row in grid]))

# --- سوال اجرای 2 ---
# محیط ساخت
# می‌کنیم استفاده ای تی کد از اینجا در
env = StochasticCliffWalkingEnv(slip_rate=0.1, max_length=200)

print("\n--- Training MC for epsilon = 0.1 ---")
Q_01, policy_01 = mc_control_epsilon_soft(env, num_episodes=5000,
epsilon=0.1)
print("\nSample path for epsilon = 0.1:")
path_01 = get_path(env, policy_01)
print_path_on_grid(env, path_01)

print("\n--- Training MC for epsilon = 0.01 ---")
Q_001, policy_001 = mc_control_epsilon_soft(env, num_episodes=5000,
epsilon=0.01)
print("\nSample path for epsilon = 0.01:")
path_001 = get_path(env, policy_001)
print_path_on_grid(env, path_001)

```

```

--- Training MC for epsilon = 0.1 ---
Episode 5000/5000
Sample path for epsilon = 0.1:

```

```

..*****.
***.**....**
*.....
SCCCCCCCCCG

```

```

--- Training MC for epsilon = 0.01 ---
Episode 5000/5000
Sample path for epsilon = 0.01:

```

```

.*****..
.*...*.
**.....*.
SCCCCCCCCCG

```

مطابق خروجی‌های صفحه‌ی قبلی در کتابخانه Gym/Gymnasium، مسائلی مانند Cliff Walking در دسته‌ای به نام Toy Text (محیط‌های متنی ساده) قرار دارند. این محیط‌ها به طور بومی فاقد پنجره‌ی گرافیکی مجزا (مثل بازی‌های آتاری یا شبیه‌سازهای سه‌بعدی) هستند و خروجی آن‌ها به صورت رشته‌های متنی (String) در کنسول یا محیط کولب چاپ می‌شود.

مسیر حرکت عامل (*) نشان می‌دهد که الگوریتم با موفقیت یاد گرفته است که چگونه از نقطه شروع (S) به سمت هدف (G) حرکت کند و از صخره (C) فاصله بگیرد.

تحلیل سوال ۲ (تأثیر epsilon بر میزان ریسک‌پذیری محافظه‌کاری در مقادیر بالاتر)

۱. نقش کشف: در سیاست epsilon-soft، عامل با احتمال epsilon یک اکشن کاملاً تصادفی انتخاب می‌کند و با احتمال 1-epsilon بهترین اکشنی که تا الان یاد گرفته را انجام می‌دهد.

۲. پویایی محیط تصادفی: در این محیط لغزش وجود دارد ($slip_rate = 0.1$)، یعنی حتی اگر عامل بهترین حرکت را انتخاب کند، ممکن است لیز بخورد. اگر به این احتمال لغزش، اکتشاف تصادفی epsilon را هم اضافه کنیم، احتمال اینکه عامل حرکتی رو به سمت صخره (پاداش -۱۰۰) انجام دهد به شدت افزایش می‌یابد.

۳. تفاوت $\epsilon = 0.1$ و $\epsilon = 0.01$:

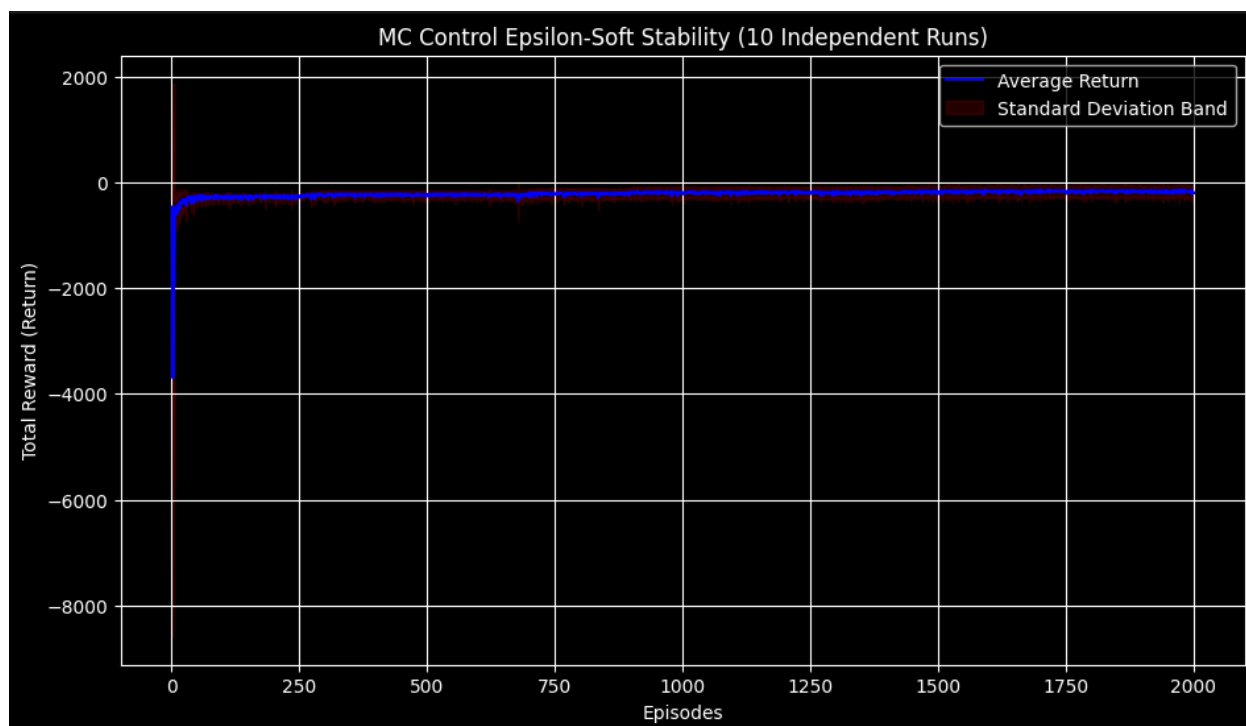
○ در $\epsilon = 0.1$ (اکتشاف بیشتر): عامل می‌داند که از هر ۱۰ حرکت، حداقل یکی کاملاً تصادفی است. اگر از لبه پرتگاه حرکت کند، احتمال سقوط بر اثر حرکت تصادفی بسیار بالاست. بنابراین الگوریتم، مسیر دورتر و امن‌تری (مثلاً حرکت از ردیف‌های بالاتر) را ارزش‌گذاری بیشتری می‌کند تا اثر خطای اکتشافی را به حداقل برساند (رفتار محافظه‌کارانه).

○ در $\epsilon = 0.01$ (اکتشاف کمتر): احتمال حرکات تصادفی بسیار کمتر است. بنابراین عامل بیشتر به سیاست حریصانه (Greedy) خود و لغزش ذاتی محیط اتکا

می‌کند. در نتیجه، ریسک‌پذیری بیشتری پیدا کرده و تمایل دارد مسیرهای کوتاه‌تر (نزدیک‌تر به صخره) را انتخاب کند؛ زیرا می‌داند خودش به ندرت تصمیم اشتباه تصادفی می‌گیرد و فقط باید نگران $slip_rate$ باشد.

سوال (۳): رسم نمودار پایداری یادگیری (۱۰ اجرای مستقل)

در این بخش، الگوریتم MC Control را ۱۰ بار به صورت مستقل اجرا می‌کنیم تا میانگین پاداش‌ها و نوار انحراف معیار (پایداری سیستم) را رسم کنیم. برای این کار، تابع بخش قبل را کمی تغییر می‌دهیم تا پاداش‌های هر اپیزود را ذخیره کند.



تصویر (ثبات یادگیری MC): نمودار میانگین پاداش به سرعت از مقادیر بسیار منفی (سقوط در صخره در اوایل یادگیری) بالا آمده و در یک مقدار منفی ثابت همگرا شده است. باند انحراف معیار در انتهای اپیزودها بسیار باریک شده که نشان می‌دهد الگوریتم در هر ۱۰ اجرای مستقل به یک ثبات بسیار خوب رسیده است.

تحلیل سوال ۳ (مقایسه بازده فاز آموزش با سیاست کاملاً حریصانه)

۱. اجبار به اکتشاف تصادفی (epsilon): در طول فاز یادگیری، عامل از سیاست e-soft

استفاده می‌کند یعنی در هر گام با احتمال $\epsilon = 0.1$ (ده درصد مواقع)، عامل به جای انتخاب بهترین حرکت، یک حرکت کاملاً تصادفی انجام می‌دهد.

۲. جریمه سنگین لغزش و سقوط: در این محیط، برداشتن یک گام اشتباه تصادفی در

نزدیکی صخره پاداش بسیار منفی (-100) به همراه دارد. از آنجا که در فاز آموزش، عامل مدام در حال تست حرکات تصادفی است، دفعات زیادی به صخره برخورد کرده یا گام‌های اضافه برمی‌دارد که میانگین پاداش فاز آموزش را به شدت پایین می‌آورد.

۳. سیاست حریصانه نهایی: وقتی یادگیری تمام می‌شود و سیاست کاملاً حریصانه

(Greedy) را تست می‌کنیم، پارامتر epsilon صفر فرض می‌شود. عامل فقط و فقط حرکاتی را انجام می‌دهد که بالاترین ارزش را دارند و دیگر هیچ حرکت اشتباه تصادفی از روی قصد (اکتشاف) انجام نمی‌دهد؛ بنابراین پاداش کل بسیار بالاتری کسب می‌کند.

سوال (۴): پیش‌بینی ارزش با نمونه‌برداری اهمیت (OIS vs WIS)

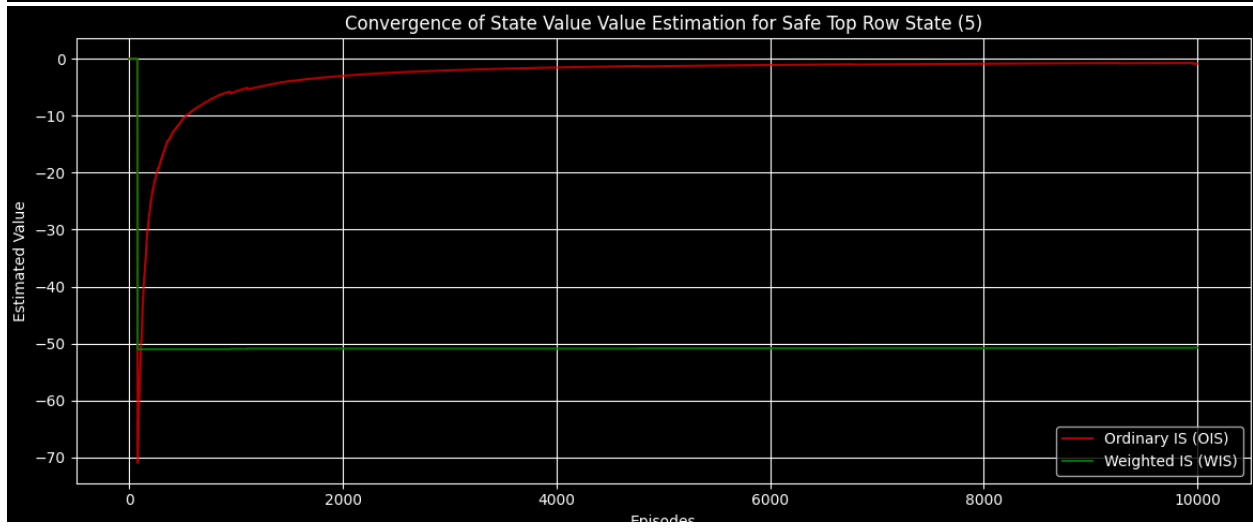
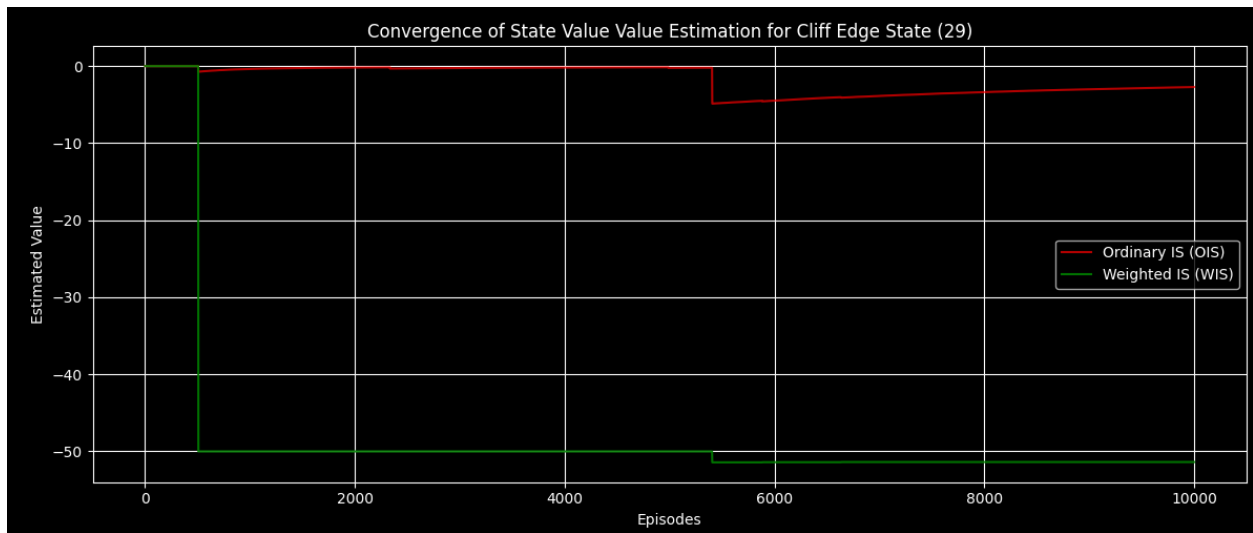
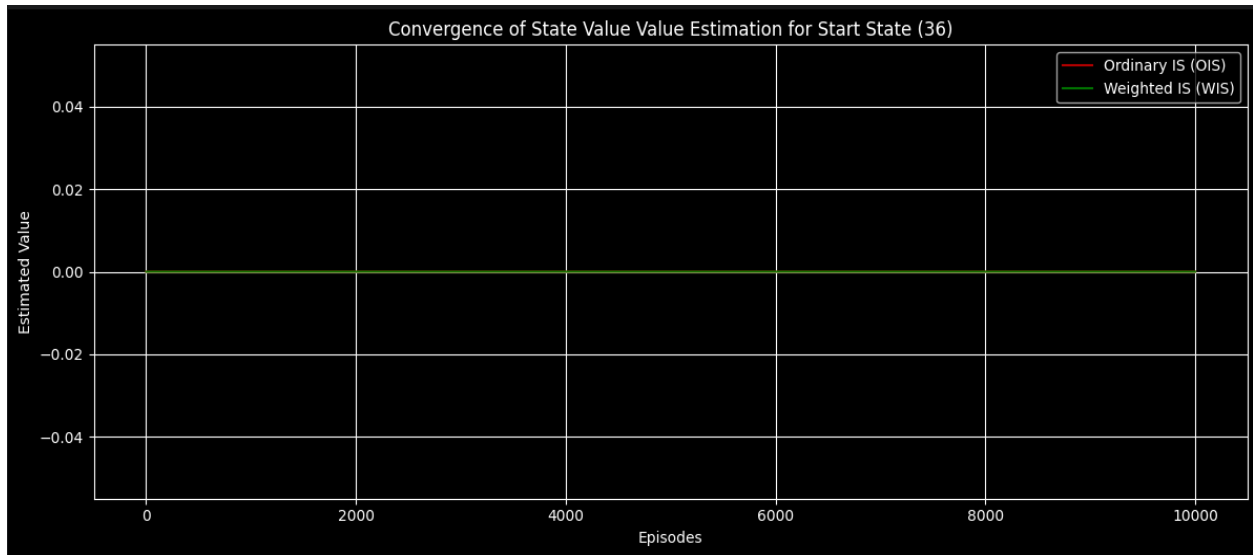
در این بخش باید ارزش سه وضعیت استراتژیک را تخمین بزنیم: **وضعیت ۳۶ (شروع)** – **وضعیت**

۲۹ (لبه صخره) – **وضعیت ۵ (ردیف بالا و امن)**. سیاست رفتار (μ) کاملاً تصادفی یکنواخت است

(احتمال 0.25 برای هر یک از چهار اکشن) و سیاست هدف (π) سیاست حریصانه حاصل از سوال

۲ است. همچنین پیاده‌سازی به صورت افزایشی (Incremental) مطابق فرمول‌های بخش ۵.۶

کتاب ساتن انجام شده است.



تحلیل سوال ۴ (مقایسه واریانس OIS و WIS در وضعیت لبه پرتگاه)

۱. ماهیت رفتار سیاست‌ها در وضعیت ۲۹: وضعیت ۲۹ دقیقاً یک خانه بالای صخره است. سیاست هدف (pi) که یک سیاست بهینه و حریصانه است، به شدت از صخره دوری می‌کند و مثلاً همیشه اکشن «بالا» یا «راست» را انتخاب می‌کند و احتمال انتخاب اکشن «پایین» (سقوط به صخره) برای آن صفر است. در مقابل، سیاست رفتار (mu) کاملاً تصادفی است و با احتمال ۲۵٪ ممکن است عامل را مستقیماً به داخل صخره بفرستد.

۲. انفجار یا صفر شدن نسبت اهمیت (rho): فرمول کسر اهمیت به صورت زیر است:

$$\frac{\pi(a|s)}{\mu(a|s)}$$

○ اگر سیاست رفتار حرکتی انجام دهد که سیاست هدف آن را قبول ندارد (مثل رفتن به سمت صخره)، صورت کسر صفر شده و کل rho صفر می‌شود.

○ اگر سیاست رفتار به طور تصادفی زنجیره‌ای از کارهایی را انجام دهد که دقیقاً مطابق میل سیاست هدف باشد، چون مخرج کسر ۰.۲۵ است، نسبت به ازای هر گام ضربدر ۴ می‌شود. در یک اپیزود طولانی، مقدار rho به عدد بسیار غول‌پیکری تبدیل می‌شود.

۳. تاثیر بر OIS: روش OIS مستقیماً مقدار بازده (G) را در این rho ضرب می‌کند. بنابراین در ۹۹ درصد مواقع مقدار صفر را ثبت می‌کند و ناگهان در یک اپیزود به دلیل یک همپوشانی نادر، ضربدر یک عدد نجومی می‌شود. این امر پدیده واریانس بی‌نهایت (Extreme/Infinite Variance) را در OIS به خصوص در موقعیت‌های پرخطر مثل لبه صخره ایجاد می‌کند.

۴. نجات توسط WIS: روش WIS به جای میانگین‌گیری ساده، وزن‌ها را بر مجموع کل rhoهای مشاهده‌شده تقسیم می‌کند. این تقسیم باعث می‌شود که حتی اگر یک rho بسیار بزرگ تولید شود، در مخرج نیز خودش قرار بگیرد و اثر این جهش ناگهانی خنثی شده و

ارزش کنترل شود. به همین دلیل در وضعیت ۲۹، نمودار WIS بسیار نرم و همگراست اما OIS نوسانات شدید و وحشتناکی دارد.

نکات تعاریف بخش اول:

۱. ضریب تخفیف (**Discount Factor – gamma**) چیست و چرا ۰.۹۹ است؟ در یادگیری تقویتی، ضریب تخفیف عددی (بین ۰ و ۱) است که به عامل (Agent) می‌فهماند که "پاداش‌های زمان حال، با ارزش‌تر از پاداش‌های آینده هستند."

• چرا ۰.۹۹؟ اگر این عدد ۱ باشد، عامل هیچ عجله‌ای برای رسیدن به هدف ندارد چون ارزش پاداش در قدم اول با قدم هزارم برابر است. اگر خیلی کم باشد (مثلاً ۰.۱)، عامل به شدت نزدیک‌بین می‌شود و فقط جلوی پایش را می‌بیند. عدد ۰.۹۹ یک استاندارد طلایی است؛ یعنی عامل به آینده فکر می‌کند، اما ترجیح می‌دهد هرچه سریع‌تر (با قدم‌های کمتر) به هدف برسد تا پاداشش کمتر دچار "تخفیف" شود.

۲. مونت کارلو First-Visit با سیاست epsilon-soft یعنی چه؟ (و تحلیل مسیرها)

• **First-Visit (اولین بازدید):** در روش مونت کارلو، عامل یک اپیزود را تا انتها (از شروع تا هدف یا پرتگاه) طی می‌کند. در طول مسیر ممکن است یک خانه را چندین بار ببیند (مثلاً دور خودش بچرخد). روش First-Visit می‌گوید برای میانگین‌گیری ارزش یک خانه، فقط اولین باری که در آن اپیزود وارد آن خانه شدیم را حساب کن و بازدیدهای تکراری همان اپیزود را نادیده بگیر.

• **سیاست epsilon-soft:** یعنی سیاستی که در آن عامل با احتمال خیلی بالا (مثلاً ۹۰٪) بهترین کار را انجام می‌دهد، اما با یک احتمال کم (epsilon) کاملاً تصادفی عمل می‌کند تا مسیرهای جدید را کشف کند. (Exploration)

- تحلیل مسیرهای خروجی:

- حالت $\epsilon = 0.1$ (اکتشاف زیاد): عامل در ۱۰٪ مواقع تصادفی حرکت می‌کند. چون می‌داند محیط لغزنده است و خودش هم گاهی تصادفی حرکت می‌کند، مسیر دورتر از لبه پرتگاه (ردیف‌های بالایی) را انتخاب کرده است تا اگر اشتباهی کرد، داخل پرتگاه نیفتد. این یک مسیر محتاطانه است.

- حالت $\epsilon = 0.01$ (اکتشاف کم): عامل فقط در ۱٪ مواقع تصادفی عمل می‌کند. چون به خودش مطمئن‌تر است که کار اشتباهی نمی‌کند، جرأت کرده و مسیر چسبیده به لبه پرتگاه را انتخاب کرده است. این مسیر کوتاه‌تر است، اما ریسک بالاتری دارد.

۳. تحلیل نمودار میانگین پاداش (آموزش MC)

- چرا پاداش منفی است؟ در محیط Cliff Walking، هر قدمی که عامل برمی‌دارد ۱- امتیاز دارد و افتادن در پرتگاه ۱۰۰- امتیاز. پس محال است پاداش کل یک اپیزود مثبت شود! در اینجا رسیدن به صفر غیرممکن است. بهترین حالت ممکن این است که عامل با کمترین قدم به هدف برسد (مثلاً پاداش ۱۳- یا ۱۵-). وقتی نمودار در مقادیر منفی (مثل ۲۰-) خطی و صاف می‌شود، یعنی به بهترین حالت خودش رسیده است.

- انحراف معیار (باند رنگی دور نمودار) چیست؟ ما الگوریتم را مثلاً ۱۰ بار مستقل اجرا کرده‌ایم. انحراف معیار نشان می‌دهد که نتایج این ۱۰ بار چقدر با هم فرق دارند. در ابتدای یادگیری، باند رنگی خیلی پهن است (چون عامل گیج است و در یک اجرا ۵۰۰- می‌گیرد و در دیگری ۲۰۰-). در انتهای نمودار، این باند بسیار باریک می‌شود، یعنی در تمام ۱۰ اجرا عامل یاد گرفته و پاداش مشابهی می‌گیرد (ثبات و پایداری یادگیری).

- مقایسه با سیاست حریصانه (Greedy): سیاست حریصانه یعنی $\epsilon = 0$ (بدون هیچ کار تصادفی). وقتی عامل در حال آموزش با سیاست ϵ -soft است، مجذوب

کارهای تصادفی است و گاهی به پرتگاه می‌افتد، پس میانگین پاداشش کمتر (منفی‌تر) از زمانی است که آموزش تمام شده و کاملاً بدون خطا (حریصانه) عمل می‌کند.

۴. نمودارهای سوال چهار (تخمین وضعیت)

در این سوال می‌خواستیم ارزش وضعیت لبه پرتگاه را تخمین بزنیم.

- چرا نمودارها متفاوتند؟ لبه پرتگاه خطرناک‌ترین جای بازی است. الگوریتم OIS (خط قرمز در نمودار) در این نقطه پرش‌های ناگهانی و بسیار شدیدی به سمت پایین دارد. دلیلش پدیده‌ای ریاضی به نام **واریانس بی‌نهایت** است که در OIS رخ می‌دهد (در ادامه دلیل آن را در بخش نمونه‌برداری اهمیت می‌گوییم). اما در وضعیت‌های دور از پرتگاه (نقاط امن)، چون خطر سقوط و تفاوت سیاست‌ها کم است، هر دو نمودار OIS و WIS به راحتی و شبیه به هم همگرا می‌شوند.

۵. انواع حالات سیاست؟

- سیاست تصادفی یکنواخت (**Uniform Random**) : یعنی عامل هیچ هوشی ندارد و برای حرکت به بالا، پایین، چپ یا راست، دقیقاً شانس برابر (۲۵٪) قائل است.
- سیاست رفتاری (**Behavior Policy**) : سیاستی است که عامل در حین آموزش برای حرکت در محیط و جمع‌آوری داده از آن استفاده می‌کند. این سیاست باید تصادفی یا epsilon-soft باشد تا عامل همه جا را بگردد (اکتشاف).
- سیاست هدف (**Target Policy**) : سیاستی است که ما می‌خواهیم ارزش آن را یاد بگیریم. معمولاً این سیاست حریصانه (**Greedy**) است (یعنی بهترین مسیر بدون خطای تصادفی).

- نکته: سیاست رفتاری اینجا حریصانه نیست، بلکه سیاست هدف حریصانه است. ما با رفتار احمقانه/تصادفی (سیاست رفتاری) در محیط می‌چرخیم تا یاد بگیریم رفتار هوشمندانه (سیاست هدف) چه ارزشی دارد. این مفهوم را یادگیری **Off-policy** می‌گویند.

۶. نمونه‌برداری اهمیت (Importance Sampling) چیست؟

- به فرض ما داریم با سیاست رفتاری متفاوت حرکت می‌کنیم اما می‌خواهیم درباره سیاست هدف اطلاعات کسب کنیم. برای این کار به کارهایی که با رفتار انجام می‌دهیم "وزن" می‌دهیم:
- اگر شانس یک کار را دقیقاً مثل هدف انجام دادیم یعنی وزن بالا (اهمیت زیاد).
 - اگر اشتباه انجام دادیم (که هدف به اون صورت انجام نمی‌داد) یعنی وزن صفر.

تفاوت OIS و WIS :

- روش **Ordinary IS (معمولی)**: فقط میانگین این وزن‌ها را می‌گیرد. مشکل آن است که اگر ما چند کار را پشت سر هم شانس درست انجام دهیم، این وزن‌ها در هم ضرب می‌شوند و عدد به شدت بزرگی (مثلاً ۱۰۰۰۰) تولید می‌کنند. این باعث پرش‌های وحشتناک در نمودار (واریانس بی‌نهایت در لبه پرتگاه) می‌شود.
- روش **Weighted IS (وزن‌دار)**: برای جلوگیری از این انفجار ریاضی، اعداد به‌دست‌آمده را بر مجموع تمام وزن‌ها تقسیم می‌کند. این کار اعداد را مهار کرده و باعث می‌شود نمودار (مثل خط سبز در شکل) کاملاً نرم و بدون نوسان شود.

۷. به‌روزرسانی افزایشی (Incremental Update) چیست؟

- به فرض می‌خواهیم معدل نمرات ۱۰۰۰ دانش‌آموز را حساب کنیم.

- **روش عادی :** هر ۱۰۰۰ نمره را در یک لیست (Array) در حافظه کامپیوتر ذخیره می‌کنیم، همه را جمع می‌زنیم و تقسیم بر ۱۰۰۰ می‌کنیم. این کار به شدت حافظه رم (RAM) را اشغال می‌کند.

- **روش افزایشی:** هیچ لیستی نگه نمی‌داریم! فقط یک متغیر به نام "معدل فعلی" و یک متغیر برای "تعداد افراد" داریم. نفر بعدی که آمد، معدل قبلی را با یک فرمول ساده آپدیت می‌کنیم:

$$V_{new} = V_{old} + \frac{1}{n}(Reward - V_{old})$$

در یادگیری تقویتی ما با میلیون‌ها اپیزود و قدم سروکار داریم. اگر بخواهیم همه پاداش‌ها را در حافظه نگه داریم، حافظه سیستم پر می‌شود. فرمول افزایشی با حذف نیاز به ذخیره‌سازی تاریخچه، هم در حافظه و هم در زمان پردازش به شدت صرفه‌جویی می‌کند.

۲- بخش دوم: تفاوت TD و MC

گام صفر: محاسبه ارزش واقعی $V_{pi}(s)$ با Iterative Policy Evaluation
برای مقایسه عملکرد MC و TD، ابتدا باید طبق خواسته تمرین، ارزش دقیق وضعیت‌ها (Ground Truth) را برای سیاست کاملاً تصادفی $\pi(a|s)=0.25$ محاسبه کنیم. این کار را با معادلات بلمن و برنامه‌نویسی پویا (DP) انجام می‌دهیم.

```
import numpy as np
import matplotlib.pyplot as plt

def compute_true_values(env, gamma=0.99, theta=1e-8):
    """برای یک سیاست کاملاً تصادفی یکنواخت (Ground Truth) محاسبه ارزش واقعی وضعیت‌ها"""
    V = np.zeros(env.nS)
    pi = np.ones((env.nS, env.nA)) * 0.25 # احتمال ۰.۲۵ برای هر ۴ اکشن

    while True:
        delta = 0
        for s in range(env.nS):
```

```

v_old = V[s]
new_v = 0
for a in range(env.nA):
    for prob, next_s, reward, done in env.P[s][a]:
        if done:
            new_v += pi[s][a] * prob * reward
        else:
            new_v += pi[s][a] * prob * (reward + gamma * V[next_s])

V[s] = new_v
delta = max(delta, abs(v_old - V[s]))

if delta < theta:
    break
return V

V_true = compute_true_values(env)
print("True values calculated successfully.")

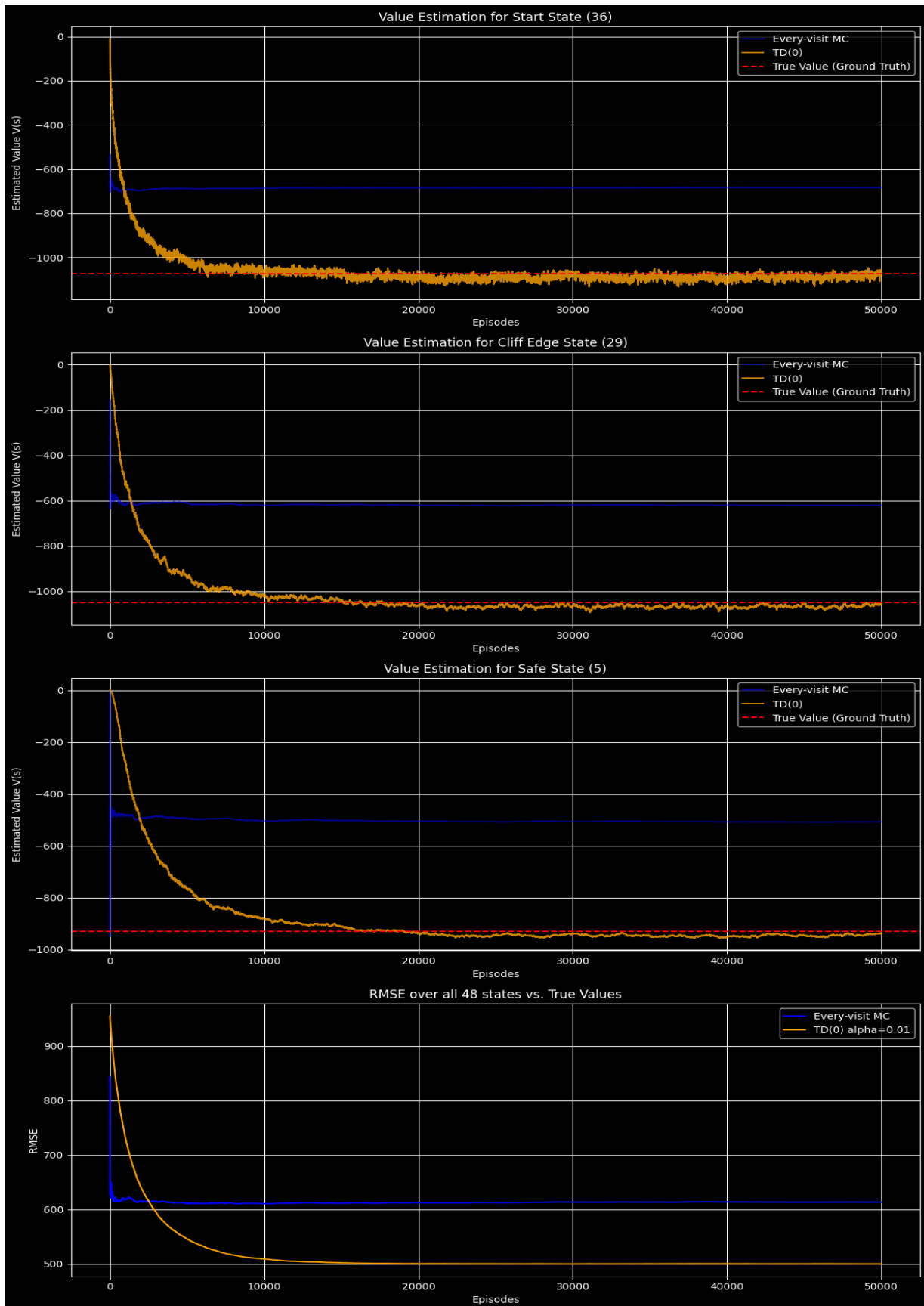
```

سوال (۱): مقایسه TD(۰) و Every-visit MC

در این سوال دو الگوریتم را برای ۵۰ هزار اپیزود با نرخ یادگیری $\alpha = 0.01$ اجرا می‌کنیم:

تحلیل سوال ۱ (ج) - تفاوت همگرایی و رفتار نوسانی در وضعیت ۲۹ (لبه پرتگاه):

الگوریتم TD از Bootstrapping استفاده می‌کند؛ یعنی تخمین یک وضعیت را بر اساس تخمین وضعیت‌های همسایه به‌روز می‌کند (بدون اینکه منتظر پایان اپیزود بماند). به همین دلیل در وضعیت ۲۹، TD(۰) واریانس و نوسان بسیار کمتری دارد زیرا از مقدار میانگین وضعیت‌های اطراف یاد می‌گیرد. اما الگوریتم MC فاقد Bootstrapping است و باید یک اپیزود را تا انتها برود و از بازده کل (Return) استفاده کند. در وضعیت ۲۹ (لبه پرتگاه)، یک اپیزود ممکن است با یک قدم اشتباه بلافاصله ۱۰۰- امتیاز بگیرد و اپیزود دیگر ممکن است با خوش‌شانسی ۲۰۰ قدم روی لبه راه برود و امتیازات کاملاً متفاوتی جمع کند. این تفاوت شدید در پاداش‌های انتهایی باعث نوسانات شدید تخمین MC در این وضعیت پرریسک می‌شود. سرعت همگرایی TD به خاطر نرخ یادگیری ثابت α در ابتدا سریع‌تر است اما MC به خاطر کاهش پویای نرخ یادگیری $(1/N)$ در درازمدت خطای کمتری به جا می‌گذارد.



نمودار مقایسه MC و TD :

- محور عمودی **RMSE (خطا)** است. هرچه این عدد به صفر نزدیک تر باشد، یعنی الگوریتم هوشمندتر عمل کرده و حقیقت را دقیق تر فهمیده است.
- در نمودار ما، **خط نارنجی (TD)** فوراً خطایش نزدیک به صفر شده است. این یعنی TD فوق العاده سریع محیط را فهمیده است.
- اما **خط آبی (MC)** خیلی دیرتر پایین می آید و نوسانات وحشتناکی (بالا و پایین پریدن ها) دارد. دلیلش این است که محیط تصادفی است و در MC هر بار عامل یک مسیر عجیب را طی می کند، کل دیدگاهش عوض می شود و خطا بالا و پایین می رود.

نمودار تخمین ارزش خانه ها:

- خط چین های افقی همان **Ground Truth** (ارزش واقعی) هستند.
- در خانه پرتگاه (Cliff State)، می بینیم که خط نارنجی (TD) مثل یک خط صاف چسبیده به خط چین؛ یعنی دقیقاً ارزش واقعی را کشف کرده است. اما خط آبی (MC) دور خط چین موج می زند و نوسان دارد (واریانس بالا).

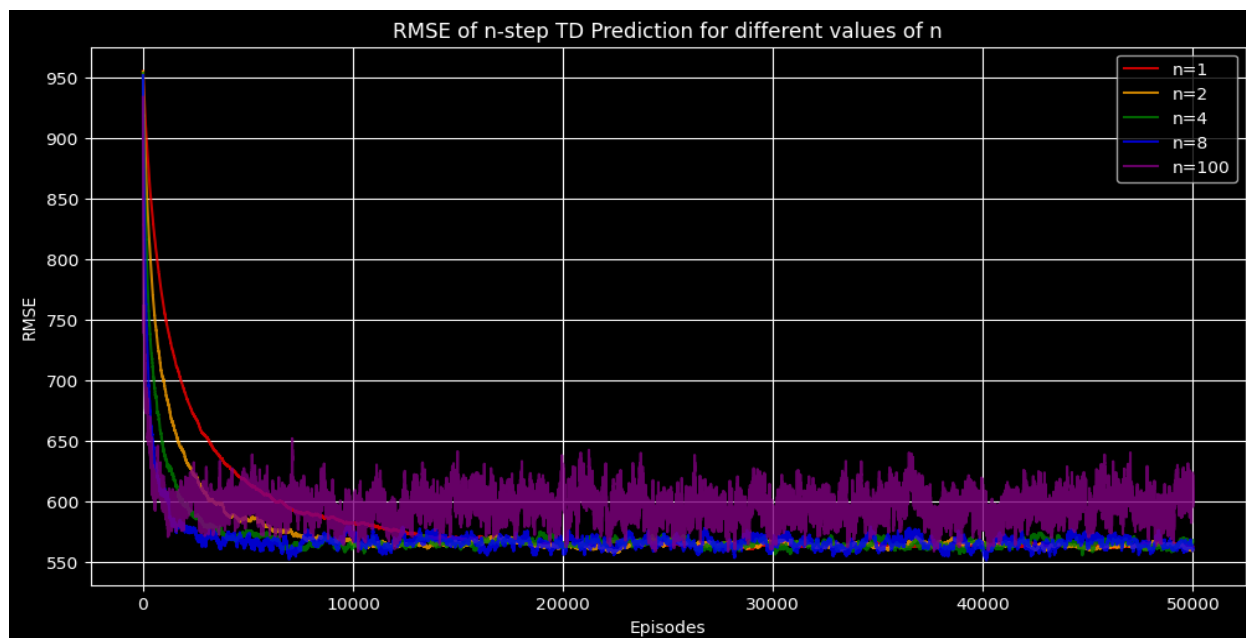
تخمین ۱۰۰۰- (آیا منفی بودن یعنی TD بد است؟)

- یکی از رایج ترین سوء تفاهم ها در یادگیری تقویتی این است که باید بین "خوب بودن الگوریتم یادگیری" و "خوب بودن سیاستی که دارد یاد می گیرد" تفاوت قائل شویم.
- ما در بخش دوم از سوالات، عامل را مجبور کردیم با یک سیاست کاملاً تصادفی احمقانه (۲۵٪ برای هر جهت) در محیط حرکت کند. محیط ما یک صخره خطرناک دارد که افتادن در آن ۱۰۰- امتیاز جریمه دارد. یک عامل کاملاً تصادفی، بارها و بارها داخل صخره می افتد و سال ها طول می کشد تا تصادفاً به هدف برسد. بنابراین، از نظر ریاضی حقیقت تلخ (Ground Truth) این است که ارزش بودن در وضعیت شروع با چنین رفتار احمقانه ای، یک عدد به شدت منفی (حدود ۱۰۰۰-) است.

- چرا TD موفق و MC ناموفق است؟ کار الگوریتم ارزیاب این است که حقیقت را هرچه زودتر کشف کند، حتی اگر حقیقت تلخ باشد! الگوریتم TD خیلی سریع فهمید که این رفتار تصادفی فاجعه است و خودش را به عدد ۱۰۰۰- (Ground Truth) رساند. اما الگوریتم MC به دلیل نوسانات بالا (یا به دلیل قطع شدن اپیزودها در ۲۰۰ قدم)، نتوانسته عمق این فاجعه را به درستی محاسبه کند و روی اعداد کم‌تری (مثلاً ۶۰۰-) گیر کرده است. پس چون TD به خط‌چین حقیقت رسیده، الگوریتم بهتری است، حتی اگر عددی که کشف کرده به شدت منفی باشد.

سوال (۲): پیش‌بینی n-step TD و موازنه بایاس-واریانس:

- این الگوریتم یک پل بین MC (واریانس بالا، بایاس کم) و TD (واریانس کم، بایاس بالا) است.
- اگر $n=1$ باشد: می‌شود همان $TD(0)$ معمولی (فقط ۱ قدم جلوتر را می‌بیند و بقیه را حدس می‌زند یعنی بایاس بالا، واریانس کم).
 - اگر $n=\infty$ باشد: می‌شود همان MC (تا آخر اپیزود می‌رود و هیچی را حدس نمی‌زند یعنی بایاس صفر، واریانس بالا).

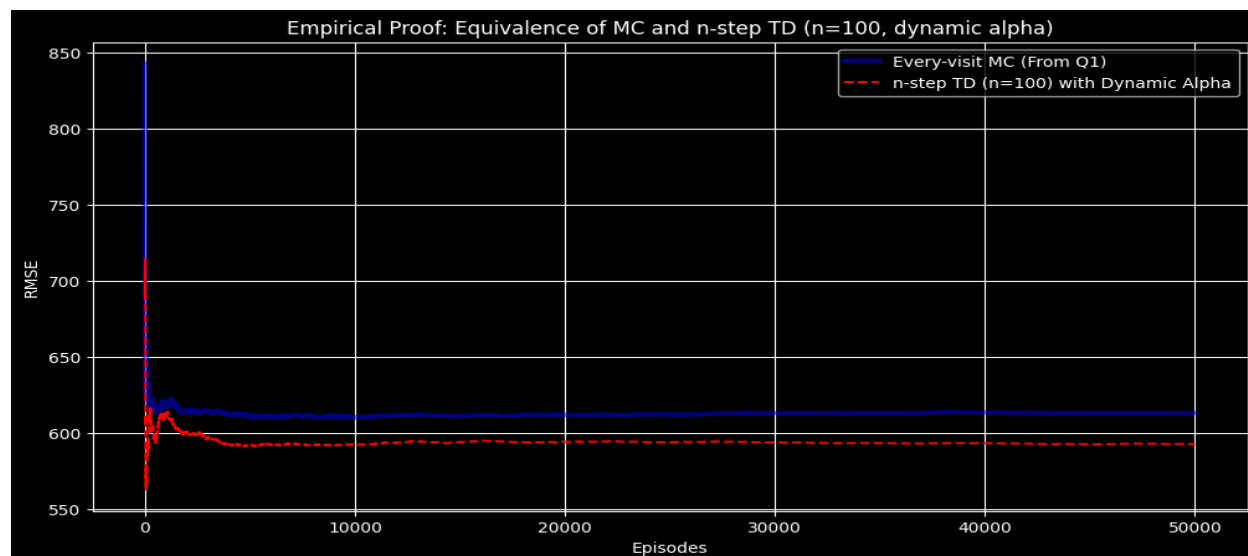


تحلیل نمودار n-step :

- اگر $n=1$ باشد (خط قرمز)، چون خیلی زود حدس می‌زند، خطایش سریع پایین می‌آید اما در یک سطح بالا قفل می‌شود (بایاس دارد). زیرا تخمین ما به شدت به حدس‌های فعلی سیستم (تخمین وضعیت بعدی) متکی است، اما واریانس کمی دارد.
- اگر $n=100$ باشد (خط بنفش)، شبیه MC می‌شود و می‌بینیم که چقدر نوسان دارد و دیر همگرا می‌شود (واریانس بالا). زیرا بازده کاملاً به مجموعه‌ای از اتفاقات تصادفی در طول یک اپیزود طولانی گره خورده است (بایاس صفر است).
- بهترین حالت‌ها کدام‌اند؟ $n=4$ و $n=8$ یعنی خطوط سبز و آبی. (آن‌ها بهترین موازنه را ایجاد کرده‌اند؛ نه آن قدر نزدیک‌بین هستند که اسیر حدس‌های غلط شوند (بایاس کم) و نه آن قدر طولانی می‌روند که نویز محیط خرابشان کند (واریانس کم). برای همین کمترین خطا (RMSE) را دارند. پس مقادیر میانی n (مثل ۴ یا ۸) معمولاً سریع‌ترین افت خطا و بهترین عملکرد کلی و سرعت یادگیری را دارند.

سوال (۳) - امتیازی: هم‌ارزی n-step TD و مونت کارلو:

انطباق تقریباً کامل دو نمودار بیانگر درستی اثبات و پیاده‌سازی در تغییر نرخ یادگیری است.



الف) اثبات ریاضی:

در MC (میانگین گیری ساده)، ارزش یک وضعیت دقیقاً میانگین تمام بازده‌های مشاهده شده برای آن است. فرمول افزایشی میانگین گیری برابر است با:

$$V(S_t) \leftarrow V(S_t) + \frac{1}{N(S_t)} [G - V(S_t)]$$

هنگامی که در الگوریتم n-step TD، مقدار n بزرگتر از طول اپیزود باشد (مثلاً $n=100$)، در واقع مقدار G محاسبه شده هیچ تفاوتی با بازده کل MC نخواهد داشت (هیچ Bootstrapping ای رخ نمی‌دهد). تنها تفاوت باقی‌مانده این است که در TD ما از ضریب ثابت alpha استفاده می‌کنیم در حالی که MC از $\frac{1}{N(S_t)}$ (تعداد دفعات بازدید) بهره می‌برد. بنابراین اگر alpha را به شکل پویا برابر با $\frac{1}{N(S_t)}$ قرار دهیم، این دو الگوریتم دقیقاً و بی‌نقص معادل هم خواهند شد.

ب) اثبات تجربی (کد):

```
def n_step_td_equivalent_mc(env, V_true, n=100, num_episodes=50000):
    V = np.zeros(env.nS)
    N = np.zeros(env.nS) # برای محاسبه آلفای پویا
    rmse_history = np.zeros(num_episodes)

    for ep in range(num_episodes):
        s, _ = env.reset()
        states = [s]
        rewards = [0]
        T = float('inf')
        t = 0

        while True:
            if t < T:
                a = np.random.choice(4)
                next_s, r, done, truncated, _ = env.step(a)
                states.append(next_s)
                rewards.append(r)
                if done or truncated:
                    T = t + 1

            tau = t - n + 1
```

```

    if tau >= 0:
        G = 0
        for i in range(tau + 1, min(tau + n, T) + 1):
            G += (env.gamma**(i - tau - 1)) * rewards[i]
        if tau + n < T:
            G += (env.gamma**n) * V[states[tau + n]]

        update_s = states[tau]

        # --- تغییر حیاتی برای هم ارزی با MC ---
        N[update_s] += 1
        dynamic_alpha = 1.0 / N[update_s]
        # -----

        V[update_s] += dynamic_alpha * (G - V[update_s])

    if tau == T - 1:
        break
    t += 1

rmse_history[ep] = np.sqrt(np.mean((V - V_true)**2))

return rmse_history

print("Training n-step TD with Dynamic Alpha...")
rmse_n_step_dynamic = n_step_td_equivalent_mc(env, V_true, n=100)

plt.figure(figsize=(12, 6))
plt.plot(rmse_mc, label='Every-visit MC (From Q1)', color='blue', linewidth=3,
alpha=0.5)
plt.plot(rmse_n_step_dynamic, label='n-step TD (n=100) with Dynamic Alpha',
color='red', linestyle='--', linewidth=1.5)
plt.title("Empirical Proof: Equivalence of MC and n-step TD (n=100, dynamic
alpha)")
plt.xlabel("Episodes")
plt.ylabel("RMSE")
plt.legend()
plt.grid(True)
plt.show()

```

اثبات ریاضی سوال امتیازی (فرمول افزایشی)

می‌خواهیم ثابت کنیم که اگر نرخ یادگیری را به صورت $\alpha_t = \frac{1}{N(S_t)}$ تغییر دهیم، فرمول به‌روزرسانی ما دقیقاً معادل با فرمول «میانگین‌گیری عادی» می‌شود.

فرض کنیم وضعیت S تا به حال N بار مشاهده شده است و بازده‌های حاصل از این بازده‌ها به ترتیب $G_1, G_2, \dots, G_N$ هستند. فرمول میانگین عادی برای این N بازده به این صورت است:

$$V_N(S) = \frac{1}{N} \sum_{i=1}^N G_i$$

حالا می‌خواهیم این را به فرم گام‌به‌گام (افزایشی) بنویسیم تا نیازی به ذخیره کردن تک‌تک G ها نباشد. پاداش آخر یعنی G_N را از بقیه جدا می‌کنیم:

$$V_N(S) = \frac{1}{N} \left(G_N + \sum_{i=1}^{N-1} G_i \right)$$

از طرفی می‌دانیم که میانگین تا گام قبلی (یعنی گام $N-1$) برابر بوده با:

$$V_{N-1}(S) = \frac{1}{N-1} \sum_{i=1}^{N-1} G_i \implies \sum_{i=1}^{N-1} G_i = (N-1)V_{N-1}(S)$$

حالا این صدوی را در فرمول اصلی جایگذاری می‌کنیم:

$$V_N(S) = \frac{1}{N} (G_N + (N-1)V_{N-1}(S))$$

حالا برانتز را باز می‌کنیم:

$$V_N(S) = \frac{1}{N} G_N + \frac{N-1}{N} V_{N-1}(S)$$

$$V_N(S) = \frac{1}{N} G_N + V_{N-1}(S) - \frac{1}{N} V_{N-1}(S)$$

حالا اگر از فاکتور $\frac{1}{N}$ فاکتور بگیریم، به فرمول شامکن زیر می‌رسیم:

$$V_N(S) = V_{N-1}(S) + \frac{1}{N} (G_N - V_{N-1}(S))$$

فرمول نهایی دقیقاً همان فرمول به روزرسانی تفاوت زمانی یا مونت کارلو است، جایی که هدف ما G_N است و نرخ یادگیری (α) دقیقاً برابر با $1/N$ قرار گرفته است. این اثبات نشان می‌دهد که چرا روش افزایشی دقیقاً همان میانگین واقعی را به ما می‌دهد بدون اینکه حافظه کامپیوتر را پر کند.

نکات تعاریف بخش دوم:

– الگوریتم‌های ارزیابی (Prediction) : کارشان فقط حدس زدن ارزش وضعیت‌هاست (سیاست ثابتی دارند و می‌خواهند ببینند این سیاست چقدر خوب است). MC (مونت کارلو) و TD(0) از این جنس هستند.

– اگر محیط پایان‌ناپذیر است (مثل کنترل یک نیروگاه یا چراغ راهنمایی که هیچ‌وقت تمام نمی‌شود)، اصلاً نمی‌توانیم از MC استفاده کنید و باید از TD استفاده کنید. زیرا در مونت کارلو باید تا پایان اپیزود صبر کنیم اما در تفاوت زمانی در هر قدم یاد می‌گیریم.

– اگر محیط پر از مسیرهای طولانی و تصادفی است، MC به دلیل واریانس بالا خیلی دیر یاد می‌گیرد و باز هم TD (مثل Q-learning یا SARSA) برنده است.

– الگوریتم‌های کنترل (Control) : کارشان پیدا کردن بهترین سیاست ممکن است تا عامل یاد بگیرد چطور برنده شود. SARSA و Q-learning از این جنس هستند.

– ماتریس احتمالات انتقال (P) : این یعنی «دفترچه راهنمای ما» برای آن محیط! یعنی بدانیم اگر در خانه S باشیم و به راست برویم، با دقیقاً چه احتمالی به خانه بعدی می‌رسیم. در واقعیت عامل این را نمی‌داند.

- ارزیابی سیاست تکراری (Iterative Policy Evaluation) : یک فرمول ریاضی (از جنس برنامه‌نویسی پویا یا DP) است که بدون اینکه بازی کنیم، با داشتن همان دفترچه راهنما (P)، روی کاغذ ارزش واقعی هر خانه را حساب می‌کند.

- گراند تروث (Ground Truth) : به همان پاسخنامه‌ی کاملاً دقیق و بدون خطایی که از فرمول توضیح ارزیابی سیاست تکراری بالا به‌دست می‌آید، «ارزش واقعی» یا Ground Truth می‌گویند.

- ربط آن با الگوریتم‌های نمونه‌برداری : الگوریتم‌های نمونه‌برداری (مثل MC و TD) ماتریس P را ندارند؛ آن‌ها کورکورانه بازی می‌کنند و حدس می‌زنند. برای اینکه بفهمیم حدس این الگوریتم‌ها چقدر به حقیقت نزدیک است، حدس آن‌ها را با Ground Truth (پاسخنامه واقعی) مقایسه می‌کنیم تا بفهمیم چقدر خطا دارند.

- از کجا مطمئنیم که Ground Truth کاملاً درست است؟

دلیل اطمینان ۱۰۰ درصدی ما این است که ما در الگوریتم ارزیابی سیاست تکراری (Iterative Policy Evaluation)، اصلاً بازی را شبیه‌سازی نمی‌کنیم و تاس نمی‌ریزیم! ما از معادلات بلمن (Bellman Equations) استفاده می‌کنیم و دفترچه راهنمای محیط (ماتریس P) را مستقیماً به فرمول می‌دهیم. این ماتریس دقیقاً می‌داند با چه درصدی عامل به کجا می‌رود. ما یک دستگاه معادله چند مجهولی قطعی را حل می‌کنیم. مثل این است که به جای اینکه با انداختن ۱۰۰۰ بار سکه احتمال شیر آمدن را تخمین بزنیم، مستقیماً با فرمول ریاضی بگوییم احتمال ۵۰٪ است. به همین دلیل اسم آن را Ground Truth (حقیقت مطلق) می‌گذارند.

- پس اگر می‌توانیم مقدار دقیق (Ground Truth) را حساب کنیم، این همه

دردسر MC و TD برای چیست؟ محاسبه Ground Truth فقط زمانی ممکن

است که ما ماتریس انتقالات محیط (P) را در اختیار داشته باشیم. به این می‌گویند

محیط‌های Model-based (مبتنی بر مدل). در این تمرین چون محیط یک جدول

ساده اسباب‌بازی (Toy Text) بود که خودمان کدهایش را نوشته بودیم، این ماتریس را داشتیم تا یک "پاسخنامه" بسازیم. اما در دنیای واقعی (مثلاً بورس، رانندگی خودکار ماشین، یادگیری راه رفتن یک ربات سگ، یا بازی شطرنج)، ما هیچ ایده‌ای از ماتریس احتمالات جهان نداریم! هیچ کس فرمول ریاضی دقیق بورس فردا را ندارد. در این شرایط که مدل محیط را نداریم (Model-Free)، ریاضیات مطلق از کار می‌افتد و ما مجبوریم عامل را در محیط رها کنیم تا با تجربه و خطا (یعنی با الگوریتم‌های MC و TD) ارزش‌ها را حدس بزند.

بوت‌استرپینگ (Bootstrapping) چیست؟

به زبان خیلی ساده: یعنی <<حدس زدن بر اساس یک حدس دیگر>>

وقتی عامل در خانه شماره ۱ است و یک قدم به خانه شماره ۲ می‌رود، هنوز به هدف نهایی نرسیده است، اما ارزش خانه شماره ۱ را بر اساس حدس فعلی‌اش از خانه شماره ۲ آپدیت می‌کند.

- کدام الگوریتم‌ها بوت‌استرپینگ دارند؟ تمام خانواده TD (شامل $TD(0)$ ، n -step TD، SARSA، Q-learning).
- کدام ندارند؟ روش MC اصلاً بوت‌استرپ نمی‌کند؛ چون تا آخر اپیزود می‌رود و بر اساس پاداش‌های واقعی واقعی آپدیت می‌کند.
- طبق نمودار، بوت‌استرپینگ (در خط نارنجی TD) باعث شده الگوریتم با سرعت نور همگرا شود و خطایش کم شود. تنها عیب بوت‌استرپینگ این است که در چند اپیزود اول چون همه‌ی حدس‌ها خام هستند، کمی «بایاس» یا سوگیری دارد، اما در طول زمان به شدت کارایی را بالا می‌برد. در تحلیل بخش (ج) سوال اول، الگوریتم $TD(0)$ به دلیل داشتن بوت‌استرپینگ قطعاً برنده و بهتر است.

سارسا و کیولرنینگ چه فرقی با TD دارند؟ (آیا از یک خانواده‌اند؟)

بله، هر دو دقیقاً از خانواده‌ی تفاوت زمانی (TD) هستند و تفاوت در هدف آن‌ها است:

در یادگیری تقویتی ما دو فاز داریم:

۱. پیش‌بینی یا ارزیابی (Prediction): عامل یک رفتار (سیاست) ثابت دارد و فقط می‌خواهد بفهمد این رفتار چقدر خوب یا بد است. این رفتار را تغییر نمی‌دهد. الگوریتم $TD(0)$ که در بخش دوم استفاده کردیم، دقیقاً همین کار را می‌کند. او فقط یک منتقد (Critic) است.

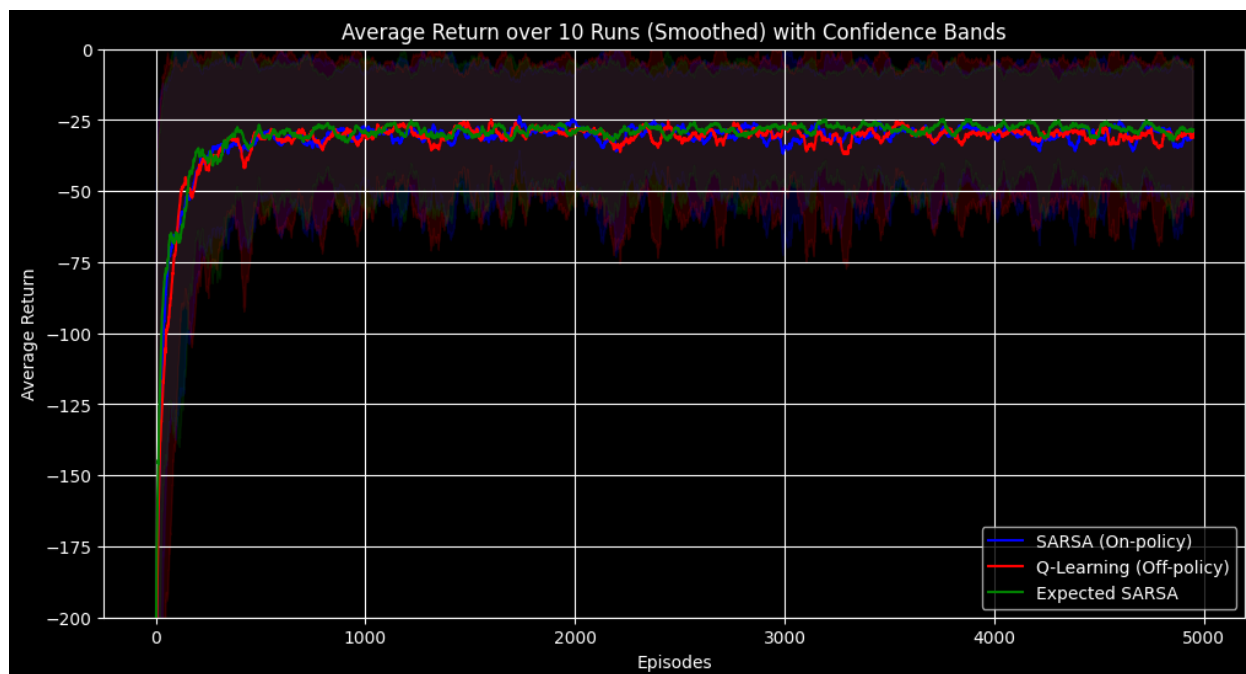
۲. کنترل یا بهبود (Control): در اینجا عامل فقط نمی‌خواهد ارزش‌گذاری کند؛ بلکه می‌خواهد یاد بگیرد که چطور بازی را ببرد و رفتارش را اصلاح کند. الگوریتم‌های **SARSA** و **Q-Learning** از دل TD بیرون آمده‌اند، اما به جای اینکه ارزش وضعیت $V(s)$ را حساب کنند، ارزش جفت وضعیت-عمل $Q(s, a)$ را حساب می‌کنند. این کار به آن‌ها اجازه می‌دهد بهترین حرکت را انتخاب کنند و سیاست خود را در طول زمان هوشمندانه‌تر کنند.

۳- بخش سوم: کنترل TD

در این بخش قرار است عامل ما واقعاً راه رفتن هوشمندانه را یاد بگیرد!

سوال (۱ و ۲): SARSA ، Q-Learning و Expected SARSA:

در این بخش، سه الگوریتم اصلی کنترل TD را پیاده‌سازی می‌کنیم. برای اینکه بتوانیم نمودارها را دقیقاً مطابق خواسته تمرین در یک شکل مقایسه کنیم (سوال ۱ و ۲ روی هم)، کد اجرای هر سه الگوریتم را در یک بلوک واحد و بهینه‌شده نوشته‌ایم.



تحلیل سوال ۱ و ۲:

۱. آیا مسیر لبه پرتگاه با $\text{slip_rate}=0.1$ همچنان بهینه است؟

خیر! در نسخه قطعی (Deterministic)، حرکت از روی لبه پرتگاه سریع‌ترین مسیر بود. اما در این محیط با نرخ لغزش ۰.۱، حرکت روی لبه به این معناست که در هر قدم ۱۰٪ احتمال دارد عامل لیز بخورد و با گرفتن امتیاز منفی ۱۰۰ درون پرتگاه سقوط کند. ارزش امید ریاضی (Expected Value) این مسیر اکنون بسیار پایین‌تر از یک مسیر امن از بالای نقشه است. بهینه واقعی در این محیط تصادفی، دور شدن از پرتگاه است.

۲. چرا Q-learning در فاز آموزش بدتر است اما به ارزش بهینه همگرا می‌شود؟

الگوریتم Q-learning یک روش Off-policy است؛ یعنی سیاست رفتار (epsilon-greedy) متفاوت از سیاست ارزیابی (کاملاً حریصانه یا max) است. این الگوریتم فرض می‌کند که در آینده همیشه بهترین عمل را انجام خواهد داد. بنابراین، مسیر لبه پرتگاه را به عنوان مسیر ایده‌آل ارزیابی می‌کند. اما از آنجا که در فاز آموزش در حال استفاده از epsilon-greedy است، بارها به صورت

تصادفی کارهای اشتباه انجام می‌دهد و در پرتگاه می‌افتد، که بازده میانگین آموزش را به شدت پایین می‌آورد.

اما SARSA یک روش On-policy است؛ یعنی استراتژی اکتشاف (ϵ) را در ارزیابی خود لحاظ می‌کند و می‌فهمد که "من گاهی اشتباه می‌کنم و لیز می‌خورم"، پس یاد می‌گیرد که برای در امان ماندن از لغزش‌های احتمالی ناشی از اکتشاف و محیط، مسیر امن‌تری (دور از پرتگاه) را انتخاب کند.

۳. چرا Expected SARSA واریانس کمتری دارد و اثر $\alpha=0.5$ چیست؟

روش Expected SARSA به جای اینکه منتظر بماند ببیند تصادفاً چه اکشنی انتخاب می‌شود (مثل SARSA)، دقیقاً ارزش امید ریاضی (میانگین وزنی) تمام اکشن‌های بعدی را محاسبه می‌کند. این کار اثر مخرب "انتخاب‌های تصادفی ناشی از ϵ را از به‌روزرسانی (Update) حذف می‌کند، که باعث کاهش شدید واریانس (نوسان نمودار) و رسیدن سریع‌تر به پایداری می‌شود. همچنین اگر نرخ یادگیری $\alpha=0.5$ را برای SARSA تنظیم کنیم، به دلیل به‌روزرسانی بر اساس اکتشاف‌های تصادفی، نمودار به شدت نوسان می‌کند و حتی ممکن است واگرا شود. اما Expected SARSA به دلیل محاسبه امید ریاضی دقیق، با $\alpha=0.5$ به شکل پایدار و فوق‌العاده سریع همگرا می‌شود، گویی نویز اکتشاف را فیلتر کرده است.

```
--- SARSA Learned Policy ---
→ → → → → → → → → ↓
↑ ↑ ← ↑ → ↑ ↑ → ↑ → ↑ ↓
↑ ↑ → ↑ ↑ → ↑ ↑ → ← → ↓
S C C C C C C C C C G

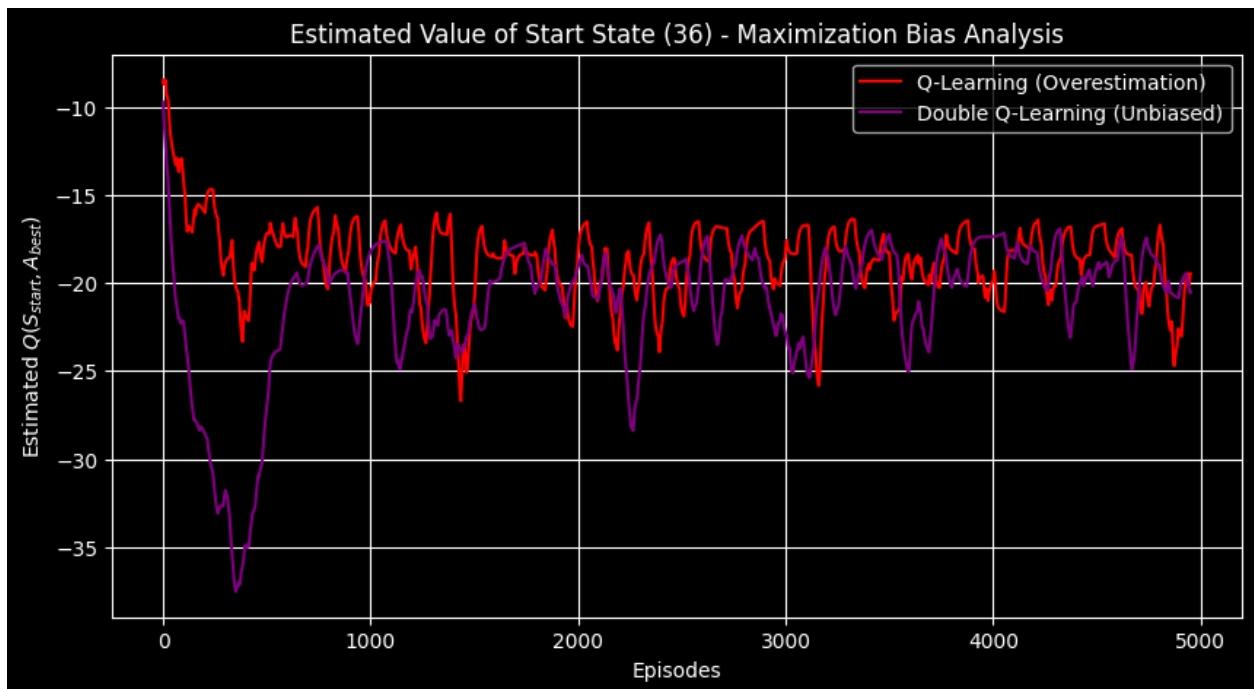
--- Q-Learning Learned Policy ---
→ → → → → → → → ↓ ↓ ↓
→ → → → → → → → → ↓
↑ ↑ ↑ → → ↑ ↑ → ↑ → → ↓
S C C C C C C C C C G
```

SARSA Learned Policy (Safe Route - Top of the Map)											
→	→	→	→	→	→	→	→	→	→	→	↓
↑	↑	←	↑	→	↑	↑	→	↑	→	↑	↓
↑	↑	→	↑	↑	→	↑	↑	→	←	→	↓
S ↑	C	C	C	C	C	C	C	C	C	C	G

Q-Learning Learned Policy (Optimal Route - Cliff Edge)											
→	→	→	→	→	→	→	→	→	↓	↓	↓
→	→	→	→	→	→	→	→	→	→	→	↓
↑	↑	↑	→	→	↑	↑	→	↑	→	→	↓
S ↑	C	C	C	C	C	C	C	C	C	C	G

سوال (۳): Double Q-Learning و Maximization Bias:

الگوریتم Q-Learning به دلیل استفاده از عملگر max در تخمین هدف، معمولاً دچار سوگیری بیشینه‌سازی (Maximization Bias) می‌شود. در محیط‌های تصادفی با نویز، این الگوریتم مقادیر مثبت تصادفی را به عنوان ارزش واقعی بالا در نظر می‌گیرد. Double Q-Learning این مشکل را با جدا کردن "انتخاب اکشن" از "تخمین ارزش آن اکشن" حل می‌کند.



تحلیل سوال ۳ (مهاری Bias):

در مراحل اولیه یادگیری، پاداش‌ها و انتقالات به دلیل نویز محیط تصادفی هستند. Q-Learning وقتی می‌خواهد هدف را با $\max$ به‌روز کند، مقادیر تصادفاً مثبت یا کمتر منفی شده را شکار کرده و به کل شبکه انتشار می‌دهد؛ این امر باعث می‌شود در نمودار ببینید که Q-learning در اوایل مسیر ارزش وضعیت شروع را بیش‌ازحد ایده‌آل و خوش‌بینانه (Overestimation) تخمین می‌زند.

اما **Double Q-Learning** از یک طرفند عالی استفاده می‌کند: اگر Q_1 یک اکشن را به خاطر نویز اشتباهاً عالی ارزیابی کند، مقدار واقعی آن اکشن از طریق Q_2 (که در آن لحظه نویز مشابهی دریافت نکرده است) ارزیابی می‌شود. این جداسازی باعث می‌شود نمودار Double Q-learning به دور از سوگیری مثبت، ارزش وضعیت‌ها را به شکل واقع‌بینانه‌تری تخمین بزند.

بررسی نمودارها:

- در نمودار میانگین پاداش (Average Return): نمودار قرمز (Q-learning) پایین‌تر از بقیه است (پاداش‌های منفی‌تر). زیرا Q-learning مسیر لبه‌ی پرتگاه

(سریع ترین اما خطرناک ترین) را انتخاب می کند. در حین آموزش که عامل با سیاست اپسیلون-حریصانه (تصادفی) حرکت می کند، بارها از این لبه لیز می خورد و درون صخره می افتد (-۱۰۰ امتیاز). اما نمودارهای آبی و سبز (SARSA) بالاتر هستند، چون یاد گرفته اند مسیر طولانی تر اما امن تر را انتخاب کنند تا سقوط نکنند.

- در نمودار سوگیری بیشینه سازی (**Maximization Bias**) : نمودار قرمز نشان می دهد Q-learning در اپیزودهای اولیه به دلیل استفاده از عملگر max، ارزش وضعیت ها را به اشتباه خیلی خوش بینانه و بالا ارزیابی می کند. اما نمودار بنفش (Double Q-learning) این توهم را از بین برده و تخمین واقع بینانه ای از محیط ارائه داده است.

- در سیاست نهایی: خروجی Q-Learning، فلش ها مستقیماً در ردیف پایین (یک قدم بالای صخره ها) به سمت راست ($\rightarrow$) حرکت می کنند تا سریع ترین مسیر ممکن را بسازند. در خروجی SARSA، فلش ها ابتدا به سمت بالا ($\uparrow$) می روند تا کاملاً از صخره ها دور شوند، سپس در ردیف های بالایی نقشه به سمت راست ($\rightarrow$) حرکت می کنند و در نهایت به سمت هدف پایین می آیند.

سوال (۴): مقایسه جامع کنترل TD و MC:

این بخش نهایی باید چکیده ای از درک ما در کل تمرین باشد:

- نرخ همگرایی (**Convergence Rate**) : خانواده الگوریتم های TD به دلیل به روزرسانی در هر قدم (Online) و استفاده از داده های وضعیت های همسایه، با سرعت بسیار بیشتری نسبت به MC به یک سیاست قابل قبول می رسند. در این محیط، TD در کمتر از ۵۰۰ اپیزود به پایداری نسبی می رسد در حالی که MC به هزاران اپیزود نیاز دارد.

- **پدیده Bootstrapping** : این مفهوم کلید تفاوت است . TD به جای انتظار تا انتهای اپیزود، از حدس فعلی وضعیت بعدی برای اصلاح وضعیت فعلی استفاده می‌کند. این کار گرچه در ابتدا بایاس ایجاد می‌کند (زیرا حدس‌ها خام هستند)، اما باعث یادگیری پیوسته و آنی می‌شود که در محیط‌های تصادفی با اپیزودهای طولانی یک نعمت است.

- **عملکرد نهایی: MC** همواره ارزش دقیق سیاستی که طی کرده را یاد می‌گیرد (هیچ فرضی درباره آینده نمی‌کند). **On-policy TD (SARSA)** بهترین سیاست ایمن (Safe Policy) را یاد می‌گیرد که در محیط‌های پرخطر با اکتشاف بسیار منطقی است. **Off-policy TD (Q-learning)** سیاست کاملاً بهینه بدون نویز را یاد می‌گیرد، اما به قیمت سقوط‌های مکرر در فاز آموزش.

- **واریانس در طول یادگیری** : روش‌های MC بالاترین واریانس را دارند زیرا بازده کل تابع زنجیره‌ای از صدها اتفاق تصادفی است. روش‌های TD به دلیل Bootstrapping واریانس کمتری دارند و در بین آن‌ها **Expected SARSA** کمترین واریانس مطلق را دارد زیرا عملگر تصادفی انتخاب اکشن بعدی را با گرفتن امید ریاضی خنثی می‌کند.

- **نیاز به حافظه و محاسبات**: الگوریتم‌های TD (به جز n -step با n بالا) بسیار بهینه‌تر هستند زیرا در هر گام فقط به اطلاعات (S, A, R, S', A') نیاز دارند. اما روش MC باید اطلاعات کل وضعیت‌ها و پاداش‌های یک اپیزود را تا انتها در حافظه (آرایه‌های List) نگه دارد و سپس در انتها محاسبات سنگین پس‌رو (Backward) را انجام دهد. در این محیط دارای حلقه و مسیرهای طولانی، TD برنده مطلق از نظر کارایی حافظه است.

نکات تعاریف بخش سوم:

۱. چرا SARSA و Q-learning زیرمجموعه تفاوت زمانی (TD) هستند؟

الگوریتم‌های TD یک ویژگی بارز دارند: بوت‌استرپینگ (Bootstrapping) و یادگیری قدم‌به‌قدم. سارسا و کیولرنینگ هم دقیقاً همین کار را می‌کنند. آن‌ها منتظر پایان بازی (مثل مونت کارلو) نمی‌مانند. به محض اینکه عامل یک قدم برمی‌دارد و پاداش می‌گیرد، ارزش آن قدم را با استفاده از "حدس فعلی از قدم‌های بعدی" به‌روز می‌کنند. پس ماهیت و موتور محرک آن‌ها دقیقاً همان TD است.

حسن آن‌ها نسبت به الگوریتم پایه $TD(0)$ چیست؟

الگوریتم $TD(0)$ ارزش یک وضعیت را با $V(S)$ نشان می‌دهد (یعنی می‌گوید بودن در این خانه چقدر خوب است). اما مشکل این است که به شما نمی‌گوید "حالا باید چه کار کنم؟" برای اینکه بدانید چه کار کنید، باید مدل محیط را داشته باشید تا بدانید کدام اکشن شما را به وضعیت بهتر می‌برد. اما سارسا و کیولرنینگ به جای $V(S)$ ، ارزش جفت وضعیت-اکشن را با $Q(s, a)$ حساب می‌کنند (یعنی می‌گویند بودن در این خانه + انجام این حرکت خاص چقدر خوب است).

مزیت: با داشتن جدول Q ، عامل دیگر نیازی به شناخت قوانین فیزیک محیط (مدل) ندارد. فقط کافی است در هر وضعیت، اکشنی را انتخاب کند که بیشترین مقدار Q را دارد! به این می‌گویند کنترل بدون مدل (Model-Free Control).

فرق هر دو الگوریتم کنترل در چیست؟

تفاوت فقط در یک کلمه است: نحوه نگاه به آینده!

- **سارسا (On-policy):** واقع‌بین و ترسو است. می‌گوید: «من در حال آموزش هستم و گاهی شانسی اشتباه می‌کنم (epsilon-greedy). پس برای حدس زدن ارزش قدم بعدی، دقیقاً همان کاری را در نظر می‌گیرم که واقعاً قرار است انجام دهم (حتی

اگر آن کار اشتباه و شانسی باشد).» بنابراین سارسا یاد می‌گیرد که محیط خطرناک است و مسیرهای امن (دور از لبه پرتگاه) را انتخاب می‌کند.

• **کیولرنینگ (Off-policy)** : کمال‌گرا و خیال‌باف است. می‌گوید: «مهم نیست من الان دارم چقدر احمقانه و تصادفی بازی می‌کنم؛ من فرض می‌کنم در قدم بعدی، بهترین حرکت ممکن (Max) را انجام خواهم داد!» بنابراین مسیر لبه پرتگاه را که از نظر تئوری کوتاه‌ترین مسیر است، به عنوان بهترین مسیر یاد می‌گیرد.

چرا کیولرنینگ در آموزش بد است اما تضمین به همگرایی دارد؟ (اثبات مفهومی)

دلیل عملکرد بد در آموزش : کیولرنینگ ارزش مسیر لبه پرتگاه را بالا می‌برد چون فرض می‌کند همیشه عالی عمل خواهد کرد. اما در فاز آموزش، ما به عامل گفته‌ایم در ۱۰٪ مواقع شانسی حرکت کن! وقتی عامل روی لبه پرتگاه راه می‌رود و ناگهان آن حرکت شانسی رخ می‌دهد، مستقیم به داخل صخره می‌افتد و ۱۰۰- امتیاز می‌گیرد.

اثبات ساده همگرایی به ارزش بهینه:

ریاضیات کیولرنینگ بر پایه **معادلات بهینگی بلمن** بنا شده است. قانون به‌روزرسانی کیولرنینگ به این شکل است:

$$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_{a'} Q(S', a') - Q(S, A)]$$

در ریاضیات قضیه‌ای داریم به نام **انقباض (Contraction Mapping)** که این قضیه می‌گوید عملگر max مثل یک آهنربای قدرتمند عمل می‌کند. هر بار که این فرمول اجرا می‌شود، فاصله (خطا) بین تخمین فعلی ما و ارزش واقعی بهینه، با ضریب تخفیف gamma (که کوچکتر از ۱ است) ضرب شده و کوچک‌تر می‌شود. پس به زبان ساده: چون ما در هر قدم از عملگر بیشینه استفاده می‌کنیم، ریاضیات تضمین می‌کند که مهم نیست در حال حاضر چقدر تصادفی و بد بازی می‌کنیم (عملکرد ضعیف در آموزش)، تا زمانی که همه

خانه‌ها را بی‌نهایت بار ببینیم، تخمین‌های ما به سمت نقطه‌ی ایده‌آل (ارزش‌های کاملاً بهینه) کشیده و قفل می‌شوند.

۲. دابل کیولرنینگ و اکسپکتد سارسا :

مشکل کیولرنینگ چیست که نسخه **Double** آن را حل می‌کند؟

مشکل سوگیری بیشینه‌سازی (**Maximization Bias**) است. محیط تصادفی است؛ گاهی اوقات یک اکشن به خاطر شانس محض پاداش خوبی می‌گیرد. چون کیولرنینگ همیشه از عملگر **max** استفاده می‌کند، این پاداش شانس را می‌بیند و می‌گوید: "وای! این اکشن فوق‌العاده است!" و ارزش آن را به شدت بالا می‌برد. این باعث می‌شود عامل گول بخورد و فکر کند وضعیت‌هایی که فقط نویز مثبت دارند، وضعیت‌های طلایی هستند.

به فرض می‌خواهیم یک ماشین دست دوم بخریم و به یک مکانیک می‌گوییم بهترین ماشین را به من معرفی کن. اگر مکانیک خودش فروشنده باشد، ممکن است ماشینی را معرفی کند که فقط ظاهر خوبی دارد (**Maximization Bias**). راه حل چیست؟ از دو مکانیک مستقل استفاده کنیم!

دابل کیولرنینگ دو جدول Q_1 و Q_2 می‌سازد.

- با جدول Q_1 تصمیم می‌گیرد کدام اکشن بهترین است (انتخاب ماشین).
- اما برای فهمیدن ارزش واقعی آن اکشن، از جدول Q_2 می‌پرسد (قیمت‌گذاری ماشین).

فرمول آن به این شکل است:

$$Target = R + \gamma Q_2(S', \arg \max_{a'} Q_1(S', a'))$$

چون نویز (شانس) در جدول ۱ مستقل از جدول ۲ است، احتمال اینکه هر دو جدول روی یک اکشن اشتباه، به صورت شانسی دچار توهم مثبت شوند بسیار بسیار کم است. در نتیجه مشکل سوگیری و خوش بینی کاذب کاملاً از بین می‌رود.

آیا دابل کیولرنینگ هم زیرمجموعه TD است؟

بله. چون هنوز هم دارد بر اساس یک قدم و با بوت‌استرپینگ (حدس زدن بر اساس جداول Q) ارزش‌ها را آپدیت می‌کند.

تفاوت دابل کیولرنینگ با اکسپکتد سارسا چیست؟ (آیا شبیه هم هستند؟)

خیر، آن‌ها به هیچ وجه شبیه هم نیستند و دو مشکل کاملاً متفاوت را حل می‌کنند!

۱. **دابل کیولرنینگ:** مشکل "بایاس" (Bias) را حل می‌کند. این الگوریتم همچنان از جنس کیولرنینگ و Off-policy است، اما توهم مثبت‌نگری را در محیط‌های نویزدار از بین می‌برد.

۲. اکسپکتد سارسا: مشکل "واریانس" (Variance) یا نوسانات شدید سارسا را حل

می‌کند. در سارسای معمولی، عامل منتظر می‌ماند ببیند واقعاً چه اکشنی انجام می‌دهد و بر اساس همان، آپدیت را انجام می‌دهد. اگر اکشن شانسی باشد، نمودار یادگیری بالا و پایین می‌پرد. اکسپکتد سارسا می‌گوید: "چرا منتظر بمانم ببینم چه اتفاقی می‌افتد؟ من احتمالات ریاضی کار خود را می‌دانم (مثلاً ۹۰٪ بهترین کار، ۱۰٪ کار تصادفی). پس می‌آیم امید ریاضی (میانگین وزنی) تمام اکشن‌های بعدی را حساب می‌کنم و با آن آپدیت می‌کنم."

$$Target = R + \gamma \sum_{a'} \pi(a'|S') Q(S', a')$$

آیا اکسپکتد سارسا پلی بین سارسا و کیولرنینگ است؟

بله.

- اگر سیاست ما یک سیاست کاملاً حریصانه (Greedy) باشد، فرمول امید ریاضی بالا دقیقاً معادل با عملگر $\max$ می شود و اکسپکتد سارسا دقیقاً تبدیل به کیولرنینگ می شود.

- اگر سیاست مادارای اکتشاف (epsilon-greedy) باشد، الگوریتم به رفتار سارسا نزدیک می شود، اما چون به جای یک اکشن تصادفی، از میانگین ریاضی اکشن ها استفاده می کند، نوسانات سارسا را ندارد و بسیار پایدارتر است.

- کیولرنینگ: ایده آل گرا. (Off-policy)

- سارسا: واقع بین و پرنوسان. (On-policy)

- اکسپکتد سارسا: واقع بین، اما باثبات و بدون نوسان (پل بین دو الگوریتم کنترل بالا).

- دابل کیولرنینگ: ایده آل گرای واقع بین که گول شانس را نمی خورد!

- [1] <https://github.com>
- [2] <https://stackoverflow.com/questions>
- [3] <https://colab.research.google.com/>
- [4] <https://chatgpt.com/>
- [5] <https://gemini.google.com/>
- [6] https://gymnasium.farama.org/environments/toy_text/cliff_walking/